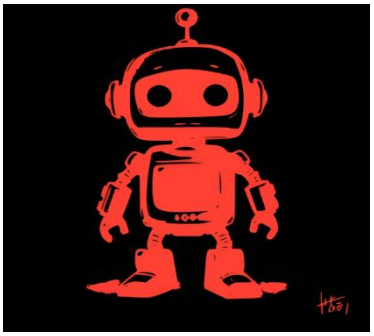

AEGIS

The Alchemists

Alchemist	Student Number
Kutlwano Tau	u24579484
Myron Wheeler	u24579565
Sambulo Zikhali	u24616592
Luyanda Ndlovu	u24580717
Lusanda Mthembu	u23602016

SRS DOCUMENT

COS301 AEGIS TEAM

UNIVERSITY OF PRETORIA

PRETORIA 0182

Contents

AEGIS DEPLOYMENT TOPOLOGY/ARRANGEMENT

The AEGIS system uses a Distributed Cloud Architecture by exploiting production-ready free-tier platforms. This architecture isolates the presentation layer, application orchestration backend, data persistence, and untrusted code execution sandbox to ensure decoupling and modularity.

COMPONENT SELECTION & COMPARATIVE ANALYSIS

Frontend Hosting: Vercel

- **Approach**: Serverless deployment by using Vercel's Edge Network.
- **Alternatives Considered:** Self-hosting on a traditional Linux server, AWS S3 and CloudFront.
    - **Justification vs. Alternatives:** AWS S3 and CloudFront offer enterprise scaling; their configuration carries variable data-egress financial risks. Vercel serves static web assets through a global Content Delivery Network (CDN) straight out of the box without any cost. This guarantees fast initial page loads globally.
- **Non-Functional Requirement Alignment:**
    - **Scalability & Concurrency:** The CDN design can handle traffic spikes with many users without UI latency.
    - **Maintainability:** Inherit GitHub integration triggers automatic compilation and deployment upon code merges, meeting the required **deployment window requirement.**

BACKEND ORCHESTRATION LAYER: FASTAPI AND PISTON ON RENDER

- **Approach:** Containerised deployment of the harmonized application backend and code execution sandbox environment on Render's managed Cloud Application Platform.
- **Alternatives Considered:** Koyeb (Deprecated Free Tier), Self-Hosted Oracle Cloud Infrastructure (OCI) Dedicated Virtual Machine
    - **Justification vs. Alternatives:** Since Koyeb's free-tier structure was deprecated, Render was selected to maintain a zero-budget layout

without introducing massive DevOps complexity of self-hosted single-node container orchestration or reverse proxy configuration. While a dedicated virtual machine avoids resource constraints, it demands cumbersome maintenance and custom script automation. Render shifts the operational complexity onto a fully managed infrastructure, accepting specific, predictable performance trade-offs in exchange for continuous deployment automation.

- **Non-Functional Requirement Alignments &Trade-offs:**
    - **Performance:** To respect the budget constraints, the backend accepts Render's free tier sleep mechanism, which suspends the container instance after 15 minutes of inbound inactivity. Therefore, the 2-second target response time applies to an active instance; initial requests will experience a 30 - to 50-second cold-start delay.
    - **Concurrency & Scalability:** The host cluster imposes a strict 512MB RAM resource ceiling. To handle concurrent user actions without prompting Out-Of-Memory (OOM) kernel crashes, the runtime application relies on FastAPI's cooperative asynchronous event-loop architecture to maximise processing efficiency under limited memory allocation
    - **Resiliency:** System recovery bypasses fragile custom scripting by depending on Render's native deployment history mechanisms. In the event of an unexpected runtime failure or unstable deployment, rollbacks are executed through the platform's control plane to instantly reroute production traffic back to the cached, last-known-stable container artifact. This yields a recovery execution. Time of under 60 seconds

DATA PERSISTENCE LAYER: SUPABASE (MANAGED POSTGRESQL)

- **Approach:** Cloud-managed PostgreSQL instance by Supabase.
- **Alternatives Considered:** Self-hosted PostgreSQL via Docker, SQLite.
    - **Justification vs. Alternatives:** SQLite does not have the required concurrent write capabilities. Self-hosting PostgreSQL on the free tier heavily consumes memory, risking Out-Of-Memory (OOM) crashes. Supabase provides an isolated database environment at no cost.
- **Non-Functional Requirement Alignment:**
    - **Security:** Natively, Supabase manages **AES-256 encryption at rest and SSL encryption in transit**, satisfying data privacy requirements.
    - **Scalability & Concurrency:** Includes Supervisor for database connection pooling, preventing connection exhaustion when many concurrent users execute database queries.

OPERATIONAL DEPLOYMENT SPECIFICATIONS

ENVIRONMENT PARITY CONFIGURATION

The system isolates operational contexts using Git branch tracking:

- **Development Environment:** Configured locally via the team's individual workstations by using Docker Compose containers pointing to a separate Supabase local/dev schema.
- Production Environment: Triggered automatically via GitHub Webhooks. Any pull requests merged into the main branch initiate an immediate secure cloud compilation and update the live endpoints on Vercel and Koyeb with no manual intervention.

INFRASTRUCTURE AS CODE & REPRODUCIBILITY

The deployment ecosystem is documented and defined via a root-level **docker-compose.yml** file to prevent configuration drift and satisfy reproducibility.

Since Piston does not make use of additional tracking databases, the configuration remains simple, allowing any evaluator to clone the repository and run a local mirror with a single command: **docker compose up –build**.

SECRETS MANAGEMENT PROTOCOL

Credentials, connection strings, and cryptographic keys are strictly excluded from version control tracking:

- An explicit template file **(.env.example)** is maintained publicly in the root directory to specify structural dependencies.
- The production **.env** configuration file is directly attached to the system's **.gitignore** rules.
- Production keys are injected into application memory at runtime through Vercel and Koyeb's secure Web UI Environment Panels, ensuring that no plain-text storage exists in code repositories.

Automated Rollback Engineering

In the event of an unexpected failure or unstable deployment during runtime evaluations, a repeatable rollback strategy is utilized:

- **Edge Layer (Frontend - Vercel):** Vercel's Dashboard retains immutable historical build images tied directly to their respective Git commit hashes. A rollback is executed instantly by designating a previous stable SHA as the active production target (recovery time: < 10 seconds). Thus, our system utilizes Vercel's native **Image Tag Pinning.**
- **Application Layer (Backend – Piston & Render):** Render's native build history is leveraged as rollbacks are executed via Render's control plane by instantly rerouting production traffic back to the cached, last-known-stable container image artifact containing both the FastAPI logic and Piston engine (recovery time < 60 seconds). Thus, our system utilizes Render's native **Immutable Artifact Rollback.**

FUTURE DEPLOYMENT STRATEGY

The current deployment architecture was selected primarily because of the project's limited development timeline and budget constraints. By utilizing free-tier cloud platforms (Vercel, Render, and Supabase), the team was able to rapidly develop, test, and deploy the system without incurring infrastructure costs while meeting the project's functional and non-functional requirements.

Following the completion of Demo 2, the team intends to migrate the frontend and backend to AWS (Amazon Web Services) while continuing to operate within the AWS Free Tier and AWS educational credits. This migration is intended to consolidate all infrastructure under a single cloud provider, simplifying deployment, security, and maintenance.

These are the intended migrations:

- Vercel Frontend – Amazon S3
- Render Backend – Amazon EC2
- Secret Management – AWS Secret Manager

DEPLOYMENT DIAGRAMS

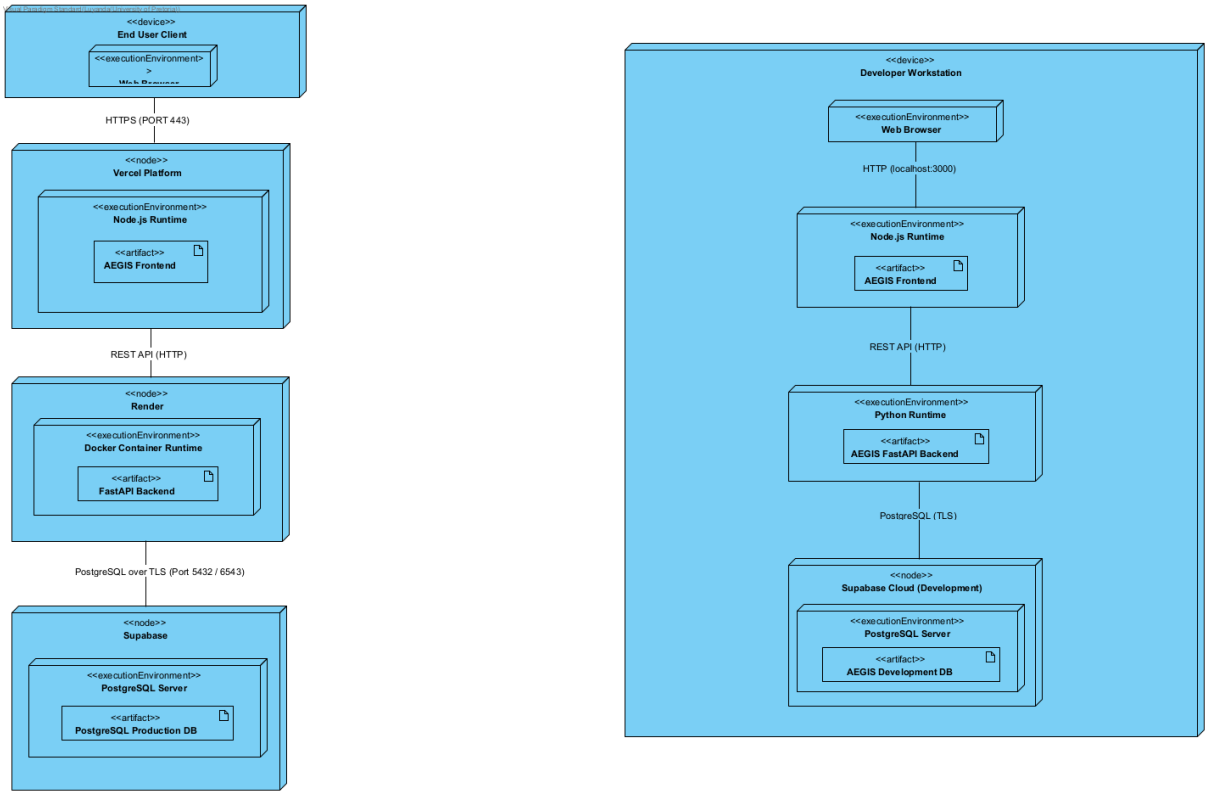

Figure 1: Deployment Diagrams

The diagram on the left in Figure 1 depicts the production deployment of AEGIS. The frontend is deployed on the Vercel platform and communicates with the FastAPI backend hosted on Render over HTTPS. The backend connects to the

production PostgreSQL database hosted on Supabase via encrypted PostgreSQL connections.

The diagram on the right in Figure 1 depicts the development deployment. Developers run the frontend and backend locally using the Node.js and Python runtimes, connecting to an isolated development PostgreSQL database hosted on Supabase. This separation ensures that development activities, testing, and database migrations do not corrupt or affect production data.